

Найди и устрани серверные уязвимости, которые могут привести к:

- несанкционированному доступу к пользовательским, клиентским или инфраструктурным данным;
- обходу аутентификации;
- обходу авторизации;
- повышению привилегий;
- выполнению действий от имени другого пользователя;
- доступу к административным или support-функциям;
- SSRF и доступу к внутренней инфраструктуре;
- утечке токенов, секретов, ключей и конфигурации;
- выполнению пользовательского кода или скриптов в доверенном интерфейсе;
- компрометации стороннего продукта, встроенного в сервис;
- массовой рассылке сообщений, писем или вредоносных файлов.

1. Аутентификация

Проверь:

- нельзя ли войти без корректной проверки пароля, токена, OTP или сессии;
- валидируется ли подпись JWT и запрещены ли небезопасные алгоритмы;
- проверяются ли `exp`, `nbf`, `iss`, `aud` и назначение токена;
- нельзя ли использовать access token как refresh token и наоборот;
- инвалидируются ли сессии после выхода, смены пароля и блокировки аккаунта;
- защищены ли процессы восстановления пароля и смены email;
- невозможно ли перебрать OTP, коды подтверждения и reset-токены;
- токены не передаются через URL, query string, referer, логи или сообщения об ошибках;
- нельзя ли подменить пользователя через заголовки, cookie, параметры запроса или данные клиента.

Все решения об идентичности пользователя должны приниматься только на сервере.

2. Авторизация и IDOR/BOLA

Для каждого endpoint проверь:

- есть ли серверная проверка прав;
- проверяется ли принадлежность объекта текущему пользователю или организации;
- нельзя ли заменить `user_id`, `account_id`, `organization_id`, `project_id`, `file_id`, `order_id` или другой идентификатор;
- невозможно ли читать, изменять, удалять или экспортировать чужие данные;
- невозможно ли вызвать административную функцию напрямую через API;
- нельзя ли выполнить действие, отсутствующее в пользовательском интерфейсе, без отдельной серверной проверки;
- проверяются ли права на вложенные объекты;
- нельзя ли получить расширенные поля через изменение параметров, GraphQL-запроса или сериализации;
- исключена ли mass assignment уязвимость.

Не доверяй ролям, идентификаторам, флагам и разрешениям, пришедшим от клиента.

Используй deny-by-default. Каждая операция должна проверять:

1. кто выполняет действие;
2. к какому tenant или организации он относится;
3. имеет ли он право на конкретное действие;
4. имеет ли он доступ к конкретному объекту.

3. Повышение привилегий

Проверь:

- нельзя ли изменить собственную роль;
- нельзя ли назначить себе admin, support, owner или service account;
- нельзя ли добавить себя в чужую организацию;
- нельзя ли изменить owner объекта;
- нельзя ли вызвать внутренний endpoint;
- нельзя ли использовать пользовательский токен для административного API;
- нельзя ли обойти ограничения через batch API, import, export, GraphQL, WebSocket или старую версию API;
- нельзя ли передать скрытые поля вроде `role`, `is_admin`, `permissions`, `status`, `verified`, `owner_id`.

Чувствительные поля должны задаваться сервером и игнорироваться при обычном пользовательском вводе.

4. SSRF

Найди все места, где сервер:

- загружает URL;
- создаёт preview;
- обрабатывает webhook;
- импортирует файл по ссылке;
- загружает изображение;
- делает callback;
- проверяет внешний ресурс;
- работает с redirect;
- обращается к пользовательскому hostname;
- проксирует запросы.

Для каждого такого места реализуй:

- allowlist разрешённых схем;
- запрет `file://`, `gopher://`, `ftp://`, `dict://` и других ненужных схем;
- запрет localhost, loopback, link-local, private, multicast и reserved IP;
- запрет IPv6 loopback и private ranges;
- защиту от альтернативного представления IP;
- повторную проверку IP после DNS resolution;
- защиту от DNS rebinding;
- повторную проверку каждого redirect;
- ограничение числа redirect;

- запрет передачи чувствительных заголовков на другой host;
- отдельный HTTP-клиент без cloud credentials;
- ограничения по timeout, размеру ответа и числу соединений;
- запрет доступа к cloud metadata endpoints;
- журналирование заблокированных попыток.

Нельзя считать защитой только проверку строки URL или наличие специального SSRF-прокси.

Если используются защитные прокси, не предполагай, что они полностью устраняют риск. Проверь, может ли пользователь:

- читать содержимое ответа;
- влиять на заголовки;
- выбирать метод;
- управлять портом;
- управлять redirect;
- обращаться к внутренним hostnames;
- получать данные через side channel.

5. Утечки данных

Проверь ответы API, логи, исключения, debug endpoints и сериализацию на раскрытие:

- персональных данных;
- токенов;
- session ID;
- API keys;
- паролей;
- reset-токенов;
- внутренних URL;
- приватных API;
- stack trace;
- SQL-запросов;
- путей файловой системы;
- переменных окружения;
- cloud metadata;
- конфигурации инфраструктуры;
- данных других пользователей или организаций.

Используй явные response DTO/schema. Не возвращай ORM-объекты напрямую.

Ошибки для клиента должны быть минимальными и безопасными. Подробности должны попадать только в защищённые внутренние логи.

6. `/metrics`, `/status`, `debug` и служебные endpoints

Найди endpoints:

- `/metrics`;
- `/status`;
- `/health`;

- `/debug` ;
- `/actuator` ;
- `/swagger` ;
- `/openapi` ;
- `/graphql` ;
- `/admin` ;
- profiler endpoints;
- trace endpoints;
- диагностические панели.

Проверь, что они:

- не доступны публично без необходимости;
- не раскрывают токены, секреты и приватные методы;
- не показывают переменные окружения;
- не раскрывают внутренние IP, hostnames и topology;
- защищены отдельной серверной авторизацией;
- ограничены по сети, если это допустимо архитектурой;
- отключены в production, если не нужны.

Не считай само наличие `/metrics` уязвимостью. Оценивай, раскрывает ли endpoint чувствительную информацию или создаёт путь дальнейшей атаки.

7. XSS и blind XSS в административных интерфейсах

Проверь все пользовательские значения, которые могут отображаться:

- в админ-панели;
- в support-интерфейсе;
- в CRM;
- в системе модерации;
- в логах;
- в тикетах;
- в аналитике;
- в уведомлениях;
- в названиях файлов;
- в email;
- в отчётах;
- в HTML preview.

Реализуй:

- context-aware output encoding;
- безопасные шаблонизаторы;
- запрет небезопасного HTML;
- sanitization только проверенной библиотекой;
- запрет `innerHTML` , `dangerouslySetInnerHTML` и аналогов без строгой обработки;
- защиту атрибутов, URL-контекста, JavaScript-контекста и CSS-контекста;
- безопасную обработку Markdown;
- защиту админских и support-сессий.

Не полагайся только на CSP. CSP является дополнительным уровнем защиты, а не заменой корректного экранирования.

8. Open Redirect

Найди параметры:

- `redirect;`
- `redirect_url;`
- `return_url;`
- `next;`
- `continue;`
- `callback;`
- `url;`
- `target.`

Разрешай только:

- относительные пути внутри приложения;
- либо явно разрешённые origin из allowlist.

Защити обработку от:

- `//evil.example;`
- encoded URL;
- двойного URL encoding;
- backslash;
- смещения схем;
- userinfo в URL;
- поддоменов злоумышленника;
- URL parser confusion;
- redirect chain.

Особенно проверь OAuth, SSO, login, logout, password reset и email verification. Нельзя допустить передачу authorization code, access token, reset token или session token на внешний домен.

9. Ограничение массовых операций

Добавь server-side ограничения для:

- регистрации;
- входа;
- восстановления пароля;
- ОТП;
- отправки email;
- отправки SMS;
- приглашений;
- комментариев;
- сообщений;
- загрузки файлов;
- генерации отчётов;

- экспортов;
- webhook;
- дорогих API-операций.

Используй сочетание:

- rate limit по пользователю;
- rate limit по IP;
- rate limit по организации;
- квоты;
- cooldown;
- idempotency keys;
- ограничение размера batch;
- очереди;
- лимиты затрат;
- защиту от повторной отправки;
- аудит аномальной активности.

Не допускай возможности массовой отправки писем регистрации, восстановления пароля, приглашений или файлов.

10. Загрузка и обработка файлов

Проверь:

- расширение;
- MIME type;
- сигнатуру файла;
- максимальный размер;
- максимальное количество файлов;
- имя файла;
- путь сохранения;
- архивы;
- вложенные архивы;
- decompression bomb;
- SVG;
- HTML;
- PDF;
- офисные документы;
- изображения;
- мультимедиа.

Требования:

- генерировать безопасное серверное имя;
- хранить файлы вне web root;
- запрещать path traversal;
- не исполнять загруженные файлы;
- отдавать потенциально опасные файлы как attachment;
- применять безопасный `Content-Type` ;
- применять `X-Content-Type-Options: nosniff` ;

- изолировать обработчики файлов;
- ограничивать CPU, RAM, disk и время обработки;
- удалять ненужные metadata, если это требуется политикой приватности;
- не передавать исходные файлы в доверенные административные интерфейсы без безопасной обработки.

Не отмечай EXIF metadata как критическую серверную уязвимость без отдельного риска, но учитывай её как вопрос приватности.

11. Инъекции

Проверь проект на:

- SQL injection;
- NoSQL injection;
- command injection;
- template injection;
- LDAP injection;
- XPath injection;
- header injection;
- CRLF injection;
- email injection;
- log injection;
- path traversal;
- unsafe deserialization;
- prototype pollution;
- server-side template injection.

Используй:

- параметризованные запросы;
- allowlist команд и аргументов;
- отказ от shell, где возможно;
- безопасные API работы с файлами;
- schema validation;
- типизацию;
- минимальные права системного пользователя;
- безопасную сериализацию.

12. Секреты и внешние сервисы

Найди:

- API keys;
- access tokens;
- private keys;
- credentials;
- DSN;
- webhook secrets;
- cloud keys;
- database URLs;

- service account credentials.

Проверь не только наличие ключа, но и его реальные полномочия.

Разделяй:

- публичные ограниченные клиентские ключи;
- серверные секреты;
- ключи сторонних сервисов;
- tracing и monitoring credentials.

Серверные секреты должны храниться в secret manager или переменных окружения, не попадать в клиентский код, репозиторий, логи и ответы API.

Добавь возможность ротации секретов.

13. Мультиотенантность

Если приложение поддерживает организации, команды, клиентов или workspace:

- всегда фильтруй данные по tenant ID на сервере;
- tenant ID должен определяться из доверенной серверной сессии;
- запрещай выбор произвольного tenant через клиентский параметр;
- проверяй межотенантный доступ во всех CRUD-операциях;
- проверяй export, search, autocomplete, attachments и background jobs;
- добавь тесты на cross-tenant data leakage;
- не используй только скрытие данных на frontend.

14. Сторонние продукты и зависимости

Проверь:

- уязвимые библиотеки;
- устаревшие зависимости;
- административные панели сторонних продуктов;
- default credentials;
- небезопасную конфигурацию;
- открытые management endpoints;
- плагины;
- SDK;
- интеграции;
- CI/CD компоненты;
- обработчики документов и изображений.

Если сторонний продукт работает внутри инфраструктуры приложения, рассматривай его компрометацию как потенциальный путь к серверу, контейнеру или данным.

Не обновляй major-версии зависимостей без анализа совместимости. Для каждого изменения укажи причину и риск.

15. Изоляция инфраструктуры

Для процессов, контейнеров и обработчиков:

- запускай без root;
- используй минимальные права;
- используй read-only filesystem, где возможно;
- ограничивай Linux capabilities;
- запрещай доступ к Docker socket;
- ограничивай сеть;
- не монтируй секреты без необходимости;
- разделяй доверенные и недоверенные операции;
- ограничивай CPU, RAM, disk и время выполнения;
- исключай общий доступ к чувствительным каталогам;
- используй sandbox для обработки недоверенных файлов и данных.

Компрометация одного обработчика не должна автоматически давать доступ ко всей инфраструктуре.

16. Проверка реального влияния

Не создавай ложные отчёты только на основании:

- отражения текста в JSON;
- некрасивого сообщения об ошибке;
- отсутствия CSP;
- возможности вызвать скрытый endpoint без вредного эффекта;
- open redirect без утечки токена или обхода доверия;
- blind SSRF без доступа к чувствительным данным или внутренним функциям;
- наличия `/metrics` без конфиденциальных данных;
- EXIF metadata без приватного контекста;
- теоретической атаки без достижимого сценария;
- проблемы, требующей полного локального доступа к устройству;
- атаки, которая зависит от недоказанной уязвимости стороннего сайта;
- self-XSS без влияния на других пользователей;
- XSS, где выполнение скрипта фактически заблокировано.

Для каждой найденной проблемы опиши доказуемую цепочку:

1. контролируемый злоумышленником ввод;
2. точка входа;
3. отсутствующая или неправильная проверка;
4. действие сервера;
5. доступные злоумышленнику данные или функция;
6. итоговое влияние;
7. необходимые права атакующего;
8. условия эксплуатации.

17. Формат результата

Сначала не изменяй код. Проведи аудит и выведи таблицу:

- Severity;
- CWE;
- файл и строка;
- уязвимый компонент;
- источник пользовательского ввода;
- опасная операция;
- сценарий эксплуатации;
- реальные последствия;
- необходимые права;
- рекомендуемое исправление;
- наличие теста.

Используй уровни:

- Critical — удалённое выполнение кода, полный обход аутентификации, компрометация инфраструктуры, доступ к критическим секретам;
- High — SSRF с чтением ответа, серьёзный IDOR, доступ к персональным или конфиденциальным данным, обход admin/support-аутентификации;
- Medium — повышение привилегий в ограниченном контексте, blind SSRF с доказанным влиянием, blind XSS в привилегированном интерфейсе;
- Low — ограниченное влияние без доступа к чувствительным данным;
- Informational — hardening без доказанной эксплуатации.

После аудита:

1. исправляй сначала Critical и High;
2. делай минимальные совместимые изменения;
3. не ломай публичные API без необходимости;
4. добавляй негативные security-тесты;
5. добавляй проверки авторизации рядом с бизнес-операцией;
6. используй существующие архитектурные паттерны проекта;
7. не создавай собственную криптографию;
8. не подавляй ошибки без безопасной обработки;
9. не заменяй уязвимый код комментариями или TODO;
10. после изменений повторно проверь весь путь данных.